

Experiment 'p_6_2_e_remove_middle_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_2_e_remove_middle_array

Natural language statement: null

Method signature: p_6_2_e_remove_middle_array(inputs: array<int>) returns (result: array<int>)

Ensures

- `result.Length == if inputs.Length % 2 == 1 then inputs.Length - 1 else inputs.Length - 2`
- `inputs.Length % 2 == 1 ==> forall i :: 0 <= i < inputs.Length / 2 ==> result[i] == inputs[i] && forall j :: inputs.Length / 2 <= j < result.Length ==> result[j] == inputs[j + 1]`
- `inputs.Length % 2 == 0 ==> forall i :: 0 <= i < inputs.Length / 2 - 1 ==> result[i] == inputs[i] && forall j :: inputs.Length / 2 - 1 <= j < result.Length ==> result[j] == inputs[j + 2]`

Requires

- `inputs.Length >= 1`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_2_e_remove_middle_array(inputs: array<int>) returns (result: array<int>)`

The method should respect the following contract:

```
requires inputs.Length >= 1, ensures result.Length == if inputs.Length % 2 == 1 then
  ↪ inputs.Length - 1 else inputs.Length - 2, ensures inputs.Length % 2 == 1 ==>
  ↪ forall i :: 0 <= i < inputs.Length / 2 ==> result[i] == inputs[i] && forall j ::
  ↪ inputs.Length / 2 <= j < result.Length ==> result[j] == inputs[j + 1], ensures
  ↪ inputs.Length % 2 == 0 ==> forall i :: 0 <= i < inputs.Length / 2 - 1 ==> result[i
  ↪ ] == inputs[i] && forall j :: inputs.Length / 2 - 1 <= j < result.Length ==>
  ↪ result[j] == inputs[j + 2]
```

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnNqWg53RqKEwSv3tRuiC1EmLb3gn

Model: gpt-5.5-2026-04-23

Created at: 1780662108

Finish reason: stop

Usage: 540 tokens in, and 6416 tokens out

```
{
  var n := inputs.Length;

  if n % 2 == 1 {
    var mid := n / 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == mid * 2 + 1;

    result := new int[n - 1];
    assert result.Length == n - 1;
    assert result.Length == 2 * mid;
    assert 0 <= mid;
    assert mid <= result.Length;

    var k := 0;
```

```

while k < result.Length
  invariant n == inputs.Length
  invariant n % 2 == 1
  invariant mid == n / 2
  invariant result.Length == n - 1
  invariant result.Length == 2 * mid
  invariant mid <= result.Length
  invariant 0 <= k <= result.Length
  invariant forall t :: 0 <= t < k && t < mid ==> result[t] == inputs[t]
  invariant forall t :: 0 <= t < k && mid <= t ==> result[t] == inputs[t + 1]
{
  if k < mid {
    result[k] := inputs[k];
  } else {
    assert k < result.Length;
    assert result.Length == n - 1;
    assert k < n - 1;
    assert k + 1 < n;
    result[k] := inputs[k + 1];
  }
  k := k + 1;
}

assert k == result.Length;
assert forall i :: 0 <= i < mid ==> result[i] == inputs[i];
assert forall j :: mid <= j < result.Length ==> result[j] == inputs[j + 1];
} else {
  assert n % 2 != 1;
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  if n == 1 {
    assert n % 2 == 1;
    assert false;
  }
  assert n >= 2;

  var mid := n / 2;
  assert n == (n / 2) * 2 + n % 2;
  assert n == mid * 2;
  assert 1 <= mid;

  var cut := mid - 1;
  assert 0 <= cut;

  result := new int[n - 2];
  assert result.Length == n - 2;
  assert result.Length == 2 * cut;
  assert cut <= result.Length;

  var k := 0;
  while k < result.Length

```

```

invariant n == inputs.Length
invariant n % 2 == 0
invariant mid == n / 2
invariant cut == mid - 1
invariant 0 <= cut <= result.Length
invariant result.Length == n - 2
invariant result.Length == 2 * cut
invariant 0 <= k <= result.Length
invariant forall t :: 0 <= t < k && t < cut ==> result[t] == inputs[t]
invariant forall t :: 0 <= t < k && cut <= t ==> result[t] == inputs[t + 2]
{
  if k < cut {
    result[k] := inputs[k];
  } else {
    assert k < result.Length;
    assert result.Length == n - 2;
    assert k < n - 2;
    assert k + 2 < n;
    result[k] := inputs[k + 2];
  }
  k := k + 1;
}

assert k == result.Length;
assert forall i :: 0 <= i < cut ==> result[i] == inputs[i];
assert forall j :: cut <= j < result.Length ==> result[j] == inputs[j + 2];
}
}

```

Final Program

```

method p_6_2_e_remove_middle_array(inputs: array<int>) returns (result: array<int>)
  requires inputs.Length >= 1
  ensures result.Length == if inputs.Length % 2 == 1 then inputs.Length - 1 else
    ↪ inputs.Length - 2
  ensures inputs.Length % 2 == 1 ==> forall i :: 0 <= i < inputs.Length / 2 ==>
    ↪ result[i] == inputs[i] && forall j :: inputs.Length / 2 <= j < result.
    ↪ Length ==> result[j] == inputs[j + 1]
  ensures inputs.Length % 2 == 0 ==> forall i :: 0 <= i < inputs.Length / 2 - 1 ==>
    ↪ result[i] == inputs[i] && forall j :: inputs.Length / 2 - 1 <= j < result.
    ↪ Length ==> result[j] == inputs[j + 2]
{
  var n := inputs.Length;

  if n % 2 == 1 {
    var mid := n / 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == mid * 2 + 1;

    result := new int[n - 1];
  }
}

```

```

assert result.Length == n - 1;
assert result.Length == 2 * mid;
assert 0 <= mid;
assert mid <= result.Length;

var k := 0;
while k < result.Length
  invariant n == inputs.Length
  invariant n % 2 == 1
  invariant mid == n / 2
  invariant result.Length == n - 1
  invariant result.Length == 2 * mid
  invariant mid <= result.Length
  invariant 0 <= k <= result.Length
  invariant forall t :: 0 <= t < k && t < mid ==> result[t] == inputs[t]
  invariant forall t :: 0 <= t < k && mid <= t ==> result[t] == inputs[t + 1]
{
  if k < mid {
    result[k] := inputs[k];
  } else {
    assert k < result.Length;
    assert result.Length == n - 1;
    assert k < n - 1;
    assert k + 1 < n;
    result[k] := inputs[k + 1];
  }
  k := k + 1;
}

assert k == result.Length;
assert forall i :: 0 <= i < mid ==> result[i] == inputs[i];
assert forall j :: mid <= j < result.Length ==> result[j] == inputs[j + 1];
} else {
  assert n % 2 != 1;
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  if n == 1 {
    assert n % 2 == 1;
    assert false;
  }
  assert n >= 2;

  var mid := n / 2;
  assert n == (n / 2) * 2 + n % 2;
  assert n == mid * 2;
  assert 1 <= mid;

  var cut := mid - 1;
  assert 0 <= cut;

  result := new int[n - 2];

```

```

assert result.Length == n - 2;
assert result.Length == 2 * cut;
assert cut <= result.Length;

var k := 0;
while k < result.Length
  invariant n == inputs.Length
  invariant n % 2 == 0
  invariant mid == n / 2
  invariant cut == mid - 1
  invariant 0 <= cut <= result.Length
  invariant result.Length == n - 2
  invariant result.Length == 2 * cut
  invariant 0 <= k <= result.Length
  invariant forall t :: 0 <= t < k && t < cut ==> result[t] == inputs[t]
  invariant forall t :: 0 <= t < k && cut <= t ==> result[t] == inputs[t + 2]
{
  if k < cut {
    result[k] := inputs[k];
  } else {
    assert k < result.Length;
    assert result.Length == n - 2;
    assert k < n - 2;
    assert k + 2 < n;
    result[k] := inputs[k + 2];
  }
  k := k + 1;
}

assert k == result.Length;
assert forall i :: 0 <= i < cut ==> result[i] == inputs[i];
assert forall j :: cut <= j < result.Length ==> result[j] == inputs[j + 2];
}
}

```

Total Token Usage

Input tokens: 540

Output tokens: 6416

Reasoning tokens: 5467

Sum of 'total tokens': 6956

Experiment Timings

Overall Experiment started at 1780662108565, ended at 1780662264698, lasting 156133ms (156.13 seconds)

Iteration #1 started at 1780662108566, ended at 1780662264697, lasting 156131ms (156.13 seconds)